I400 - Natural Language Processing - Universidad de San Andrés - 2026

Adaptive latent reasoning: PonderNet-style probabilistic *halting* over implicit chains of thought

Ana Paula Tissera, Valentino Arbelaiz Alberti, Franco Amato de Lusarreta, Naomi Couriel, Juan Cruz Giner Pulero

Universidad de San Andrés

Introduction

Latent chain-of-thought models (Coconut, CODI, SIM-CoT) replace text reasoning with continuous vectors, at a much lower cost than explicit CoT. But they fix in advance the number of latent steps K and the number of vectors per step c, applying them uniformly to every input: a two-operation problem consumes the same budget as a six-operation one. This work makes that budget **instance-adaptive**.

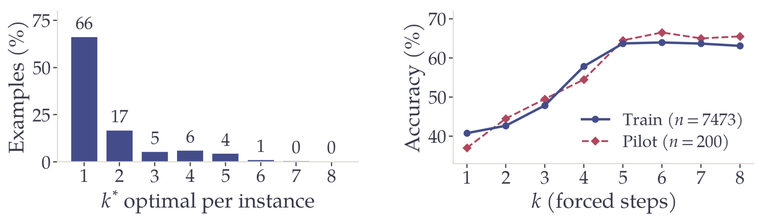
Objectives

Make the reasoning budget **instance-adaptive**, on three fronts:

- Show that the optimal budget **depends on the instance** and see whether it is predictable *upfront* from the prompt.
- Make the number of **steps** K adaptive with PonderNet-style *halting*, and characterize the accuracy–compute frontier that a fixed K does not capture **(main contribution)**.
- Explore the other budget axis: the **vectors per step** c.

How much reasoning?

We run CODI varying $k \in \{1, \dots, 8\}$ over n=7473 examples and measure k^* (the smallest k that reaches the best accuracy for each instance).



Accuracy rises 23 points up to $k \approx 5$–6 and then stabilizes. But the optimal k^* is very **imbalanced** (66 % of instances solve with k^*=1): **there is room to adapt the budget per instance**.

A classifier trained to predict k^* from frozen *embeddings* (MiniLM) does not beat the majority class (RF: 0.661; LR: 0.210). Difficulty **cannot be read *upfront***: one must decide *during* reasoning.

References

[1] Hao et al. *Training Large Language Models to Reason in a Continuous Latent Space* (Coconut), 2024.

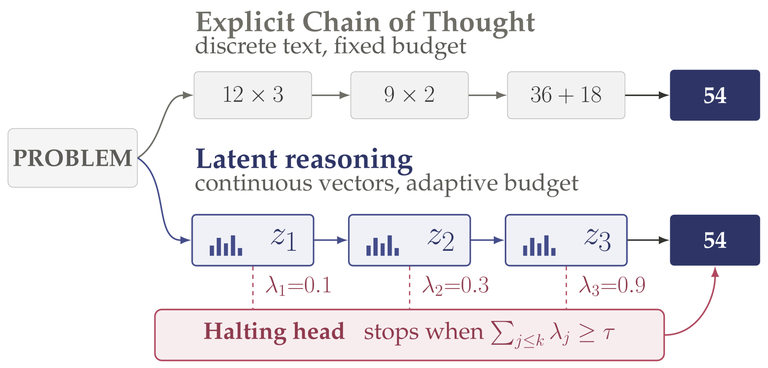
[2] Shen et al. *CODI: Compressing Chain-of-Thought into Continuous Space via Self-Distillation*, 2025.

[3] Wei et al. *SIM-CoT: Supervised Implicit Chain-of-Thought*, 2025.

[4] Banino, Balaguer and Blundell. *PonderNet: Learning to Ponder*, 2021.

Method: adaptive *halting*

The model feeds its last hidden state back as the next input (z_k, "continuous thoughts"). A single linear halting head decides, at each step, whether the accumulated reasoning is already enough to answer:



At each step, the model predicts a conditional stopping probability and a candidate answer is decoded for each prefix $k = 1, \dots, K$, forming PonderNet's stopping distribution (with an absorbing boundary at K):

$$\lambda_k = \sigma(\texttt{halt\_head}(h_k))$$
$$p_k = \left(\prod_{j<k}(1-\lambda_j)\right)\lambda_k.$$

The objective combines terms from the three methods: PonderNet (expected loss + KL regularization), SIM-CoT (per-step supervision), and CODI (distillation losses):

$$\mathcal{L} = \underbrace{\textstyle\sum_k p_k\, \mathcal{L}_{\text{ans}}^{(k)}}_{\mathcal{L}_{\text{ponder}}} + \beta\, \mathcal{L}_{\text{step}} + \gamma\, \mathrm{KL}_{\text{geom}} + \mathcal{L}_{\text{distill}} + \mathcal{L}_{\text{ref}}.$$

- The weight γ **controls the compute**: it regulates the influence of the geometric prior and, with it, how much the reasoning chain is shortened.
- At inference, reasoning stops when the accumulated stopping mass exceeds a threshold (by default 0.5); $K_{\max}$ sets the upper limit.

Per-instance geometric prior

Instead of using a global geometric prior, we make the expected budget depend on each instance. To do so, we parametrize it as an affine function of the number of operations n_i:

$$\texttt{geom\_mean}_i = \mathrm{clamp}\big(\alpha\, n_i + \beta,\ \beta,\ K_{\max}\big).$$

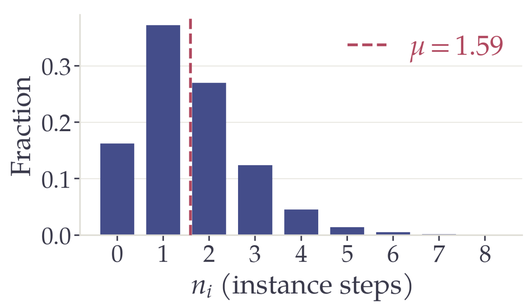


Figure. Distribution of n_i in GSM8k-Aug (μ=1.59).

Experimental setup

- **Data:** GSM8k-Aug (step-by-step reasoning, Deng et al.; ∼385k *train* examples). The training size **varies by experiment** (from the full *train* set to small subsets); the models in the results band use 100k examples.
- **Model:** GPT-2 (124M), initialized from SIM-CoT's CODI checkpoint (*warm-start*). We fine-tune the **full network** (not just adapters) together with the halting head, with a cap of $K_{\max}$=12 latent steps.
- **Evaluation:** *greedy* accuracy, single pass, and `batch_size= 1`, on the **1319-example test set** never seen in training. The epoch and halt threshold were chosen on a separate validation set (500 examples).
- **Baseline (fixed $K = 6$): 39.50 %.**

The other axis: vectors per step (c)

Each step is built with up to M sub-vectors; an MLP distills the decoder's reconstruction loss ($\hat{\mathcal{L}} = \mathrm{MLP}(h)$) and survives inference: the step ends when $\hat{\mathcal{L}}$ converges.

Init × granularity	c=1	c=2	c=3	$\Delta_{2\to3}$
warm + atomic	27.5	39.4	39.9	+0.5
warm + coarse	25.5	36.6	**40.3**	+3.7
cold + coarse	5.3	5.5	5.5	n/a

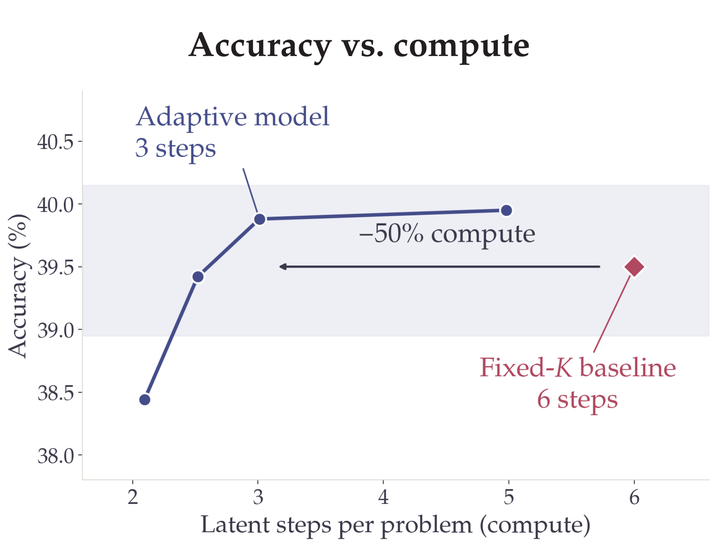
With atomic steps (one operation per step), accuracy **saturates at** $c = 2$: adaptive stopping does not improve over random. With coarse steps (variable density), the improvement reappears (adaptive > random by 2.7σ), though it is small.

The variability seems to lie in the number of steps, rather than in the complexity of each step.

Conclusions

- Learned *halting* **matches the fixed-K baseline** (≈ 39.5 % on test) while cutting **nearly half of the latent steps** (3.01 vs. 6).
- Strong difficulty tracking: **Spearman** +0.626 between operations and steps used (M2, test).
- Two informative negative results: difficulty is not readable *upfront*; the c axis has no headroom on GSM8k-Aug.
- Future work: a fair comparison training GPT-2 **from scratch** (without *warm-start*, with the full train set and 40 epochs; in progress), other backbones and tasks.

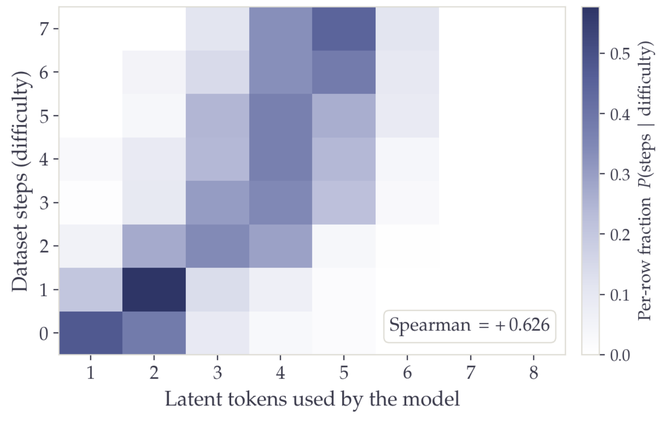
Main result: adaptive *halting* on test (n=1319): half the compute, without losing accuracy



Threshold	**M0** α1.0, γ.05	**M1** α1.0, γ.10	**M2** α0.6, γ.10
0.3	39.1 @ 3.34	39.1 @ 2.60	38.4 @ **2.10**
0.4	39.6 @ 3.86	39.4 @ 3.16	39.4 @ **2.52**
0.5	40.0 @ 4.46	39.4 @ 3.73	**39.9 @ 3.01**
0.8	40.2 @ 6.97	39.7 @ 6.39	40.0 @ 4.98

Accuracy (%) @ average latent steps. M0/M1/M2 are the same adaptive model with different regularizer intensity γ and prior slope α; fixed-K baseline: 39.5 @ 6.0.

All configurations reach similar accuracy (∼39–40 %). M2 with threshold 0.5 achieves 39.9 % using only 3.01 steps: **half** the compute of the fixed-K baseline.



Steps used vs. difficulty (M2, threshold 0.5, test): the mass follows the diagonal. **Spearman** = +0.626: compute is allocated by difficulty, not uniformly.